

BaGFN: Broad Attentive Graph Fusion Network for High-Order Feature Interactions

Zhifeng Xie, Wenling Zhang, Bin Sheng[✉], *Member, IEEE*, Ping Li[✉], *Member, IEEE*,
and C. L. Philip Chen[✉], *Fellow, IEEE*

Abstract—Modeling feature interactions is of crucial significance to high-quality feature engineering on multifield sparse data. At present, a series of state-of-the-art methods extract cross features in a rather implicit bitwise fashion and lack enough comprehensive and flexible competence of learning sophisticated interactions among different feature fields. In this article, we propose a new broad attentive graph fusion network (BaGFN) to better model high-order feature interactions in a flexible and explicit manner. On the one hand, we design an attentive graph fusion module to strengthen high-order feature representation under graph structure. The graph-based module develops a new bilinear-cross aggregation function to aggregate the graph node information, employs the self-attention mechanism to learn the impact of neighborhood nodes, and updates the high-order representation of features by multihop fusion steps. On the other hand, we further construct a broad attentive cross module to refine high-order feature interactions at a bitwise level. The optimized module designs a new broad attention mechanism to dynamically learn the importance weights of cross features and efficiently conduct the sophisticated high-order feature interactions at the granularity of feature dimensions. The final experimental results demonstrate the effectiveness of our proposed model.

Index Terms—Attention mechanism, broad learning system (BLS), feature interactions, graph neural networks.

NOMENCLATURE

Symbols Definitions and Descriptions.

m, M Size of fields and the size of training instances.

X'_m	One-hot encoding representation in the m th field.
X_i	i th instance with m field representations.
E_i	Embedding output of X_i .
n_p, n_q	p th and q th graph nodes of E_i .
$\xi(n_p, n_q)$	Edge weight between n_p and n_q .
A	Adjacency matrix with $m \times m$ edge weights.
l	Multi-hop feature fusion step.
κ^l	Input features at the step l .
$\mathcal{N}_p^{(1)}, \mathcal{N}_p^{(2)}$	First- and second-order neighborhood nodes of n_p .
$\psi_p^{(1)}, \psi_p^{(2)}$	the first- and second-order aggregations of n_p .
G_p^l	Bilinear-cross aggregation of n_p at the step l .
ϕ_p^l	Attention weight vector of n_p at the step l .
H_p^l	Interaction representation of n_p at the step l .
κ^L	High-order feature representation after L steps.
F^r, P^s	Mapping and enhanced feature nodes.
ζ	Broad attentive weights of global-aware nodes.
Γ	Final representation of cross features.
$\hat{y}, y$	Predictive result and the ground truth.
$\mathcal{L}$	Objective function with log-likelihood loss.
W	All trainable parameters in our BaGFN model.

I. INTRODUCTION

FEATURE engineering is a procedure of exploiting valid features from original data, which can significantly boost the performance of the predictive model. It is normally regarded as a central task to a variety of machine learning applications, such as recommendation system [1], computational advertising [2], and search ranking [3]. However, multifield sparse data often fail to achieve ideal feature engineering because it is quite difficult to model intricate feature interactions effectively. To that end, previous feature interaction methods [4], [5] create cross features by manually exploiting multiple features. Such feature crossing is able to disclose the implicit relationships among multifield sparse data. Nevertheless, the handcrafted feature interactions usually require extensive domain knowledge and consume a lot of time, which is hard to generate the appropriate cross features to a new domain. Therefore, in the process of feature engineering, it is crucial to automatically achieve high-order feature interactions from multifield sparse data, which is a great help to improve the accuracy of the predictive model.

Recently, a series of state-of-the-art methods have been proposed to generate cross features automatically from raw data. Factorization machine (FM) [6] is regarded as a benchmark

Manuscript received 10 February 2021; revised 11 August 2021; accepted 26 September 2021. Date of publication 8 October 2021; date of current version 4 August 2023. This work was supported in part by the National Natural Science Foundation of China under Grant 61872241 and Grant 61572316; in part by the National Key Research and Development Program of China under Grant 2019YFB1703600; and in part by The Hong Kong Polytechnic University under Grant P0030419, Grant P0030929, and Grant P0035358. (Corresponding author: Bin Sheng.)

Zhifeng Xie is with the Department of Film and Television Engineering, Shanghai University, Shanghai 200072, China, and also with Shanghai Engineering Research Center of Motion Picture Special Effects, Shanghai 200072, China.

Wenling Zhang is with the Department of Film and Television Engineering, Shanghai University, Shanghai 200072, China.

Bin Sheng is with the Department of Computer Science and Engineering, Shanghai Jiao Tong University, Shanghai 200240, China (e-mail: shengbin@sjtu.edu.cn).

Ping Li is with the Department of Computing, The Hong Kong Polytechnic University, Hong Kong (e-mail: p.li@polyu.edu.hk).

C. L. Philip Chen is with the School of Computer Science and Engineering, South China University of Technology, Guangzhou 510006, China, also with the Navigation College, Dalian Maritime University, Dalian 116026, China, and also with the Faculty of Science and Technology, University of Macau, Macau, China (e-mail: philip.chen@ieee.org).

Color versions of one or more figures in this article are available at <https://doi.org/10.1109/TNNLS.2021.3116209>.

Digital Object Identifier 10.1109/TNNLS.2021.3116209

solution for multifield sparse data in academia and industry due to its superior performance. Specifically, it transforms each feature into an embedding vector and learns second-order interactions via the inner product of pairwise features. As an improved version of FM, field-aware FM (FFM) [7] takes into account feature interactions among different fields, which can yield more powerful performance. However, the neglect of nonlinear and high-order feature interactions limits the prediction accuracy of these FM-based models. Later, numerous methods based on deep learning are proposed to probe the modeling of high-order and nonlinear feature interactions, such as neural FM (NFM) [8], Deep&Cross [2], DeepFM [9], xDeepFM [10], automatic feature interaction (AutoInt) [11], feature importance, and bilinear feature interaction network (FiBiNET) [12]. For example, NFM [8] and DeepFM [9] construct deep neural networks (DNNs) to model high-order interactions and, meanwhile, utilize FM to model low-order interactions. However, they extract cross features in a rather implicit bitwise fashion, which often lacks good model explanations and will bring negative and uncontrollable effects due to some useless feature interactions.

Unlike these implicit methods, some optimized models are capable of learning high-order interactions explicitly or the importance of features by proposing their dedicated networks. For example, Deep&Cross [2] proposes a DNN-based cross network to learn certain bounded-degree feature interactions directly; xDeepFM [10] designs compressed interaction network (CIN) to learn high-order feature interactions explicitly; AutoInt [11] constructs a self-attentive neural network to automatically learn different orders of feature interactions; FiBiNET [12] introduces a squeeze-excitation network (SENET) to learn the significance of features. Unfortunately, these optimized methods are still not sufficiently effective and explicit since they just unite feature fields to learn interactions or introduce a single module to explore the importance of features. The amorphous combination or independent attention unit lacks the competence of learning sophisticated interactions among different feature fields in a flexible and explicit manner. To better model the sophisticated high-order interactions, feature interaction GNN (Fi-GNN) [13] first introduces graph neural network (GNN) [14] to transform the modeling of feature interactions into the modeling of node interactions on a graph structure. Specifically, Fi-GNN is designed to model feature interactions by aggregating the graph node with its neighbors, and the edge weights reflect the importance of different node interactions. However, the simple aggregation manner limits the capability of cross features, and it is also incapable of learning the impact of aggregation of neighboring nodes on the node itself. In brief, the informative aggregation of graph nodes is not still comprehensive enough and flexible for sophisticated high-order interactions. Thus, we focus on how to take advantage of the GNN-based method and integrate some other advanced models to make up for its deficiency.

In this article, a novel model named broad attentive graph fusion network (BaGFN) is proposed to capture fine-grained cross features in a flexible and explicit manner. Concretely, inspired by the graph convolutional network (GCN) [15] and the broad learning system (BLS) [16], the new model transforms feature interactions into node interactions, and its architecture mainly consists of two key modules: attentive graph fusion module and broad attentive cross module. The first module constructs graph structure to generate feature representation by aggregating the information of first- and

second-order neighborhood nodes and then employs the self-attention mechanism and multihop fusion steps to learn and update the high-order interactions of graph nodes. The module cannot specify the significance of each interaction at the granularity of feature dimensions. To further refine the high-order interactions of the first module, the second module achieves a new broad attention mechanism, which can dynamically learn the weights of cross nodes that reflect the significance of different feature interactions on the final model prediction and quickly conduct high-order feature interactions at the granularity of feature dimensions. On the basis of the above two modules, our BaGFN model is able to capture the explicit influence of sophisticated high-order feature interactions among multiple fields, which will be beneficial to further improve the accuracy of the predictive model. To validate the effectiveness of our BaGFN model, we conduct extensive experiments on two benchmark click-through rate (CTR) prediction datasets (Avazu¹ and Criteo²) and a real-world industry dataset (Tobacco) with more feature fields. In summary, our main contributions are listed as follows.

- 1) *BaGFN for Modeling High-Order Feature Interactions:* The BaGFN model transforms feature interactions into node interactions and extracts fine-grained cross features among multifield sparse data in a flexible and explicit manner.
- 2) *Attentive Graph Fusion Module to Strengthen High-Order Feature Representation Under Graph Structure:* The graph-based module designs a bilinear-cross aggregation function to aggregate the graph node information, employs a self-attention mechanism to learn the impact of neighborhood nodes, and updates the high-order representation of features by multihop fusion steps.
- 3) *Broad Attentive Cross Module to Refine High-Order Feature Interactions at the Bitwise Level:* The optimized module further designs a new broad attention mechanism to dynamically learn the importance weights of cross features and efficiently conduct the sophisticated high-order feature interactions at the bitwise level.

II. RELATED WORK

A. Modeling Feature Interactions

Modeling feature interactions is an essential problem of multifield sparse data and is studied extensively in the literature. A classic algorithm is FM [6], which pays attention to second-order feature interactions by capturing the inner product of pairwise feature representations. Succeeding the success of FM, numerous variants of FM have been proposed to exploit more field information. FFM [7] presumes that each feature field possesses a separate embedding vector for each of the other feature fields. Nevertheless, these FM-based methods only consider the second-order interactions of feature fields, which neglects the nonlinear and high-order feature interactions and limits the performance of their final predictive models.

In recent years, more and more methods based on deep learning have been proposed to model high-order feature interactions. Product-based neural network (PNN) [17] applies a product layer between the embedding layer and DNN

¹<https://www.kaggle.com/c/avazu-ctr-prediction>

²<https://www.kaggle.com/c/criteo-display-ad-challenge>

layer to learn second- and high-order interactions. Different from it, NFM [8] proposes a bi-interaction pooling operation between the embedded layer and the neural network layer to model high-order and nonlinear feature interactions. Meanwhile, some researchers focus on learning low- and high-order feature interactions through a hybrid network. For instance, Wide&Deep [4] and DeepFM [9] have a shared feature embedding layer to its “wide” and “deep” component. Its “wide” part learns low-order interaction, while the “deep” part models high-order interaction. However, these DNN-based methods use rather implicit bitwise manners to conduct the high-order feature interactions, which may cause negative and uncontrollable cross features due to some useless feature interactions.

Besides, some other DNN-based methods utilize the specifically devised networks to capture high-order interactions in an explicit manner. Deep&Cross [2] introduces a novel cross network to capture the feature interactions of bounded degrees and the nonlinear feature interactions, which is further improved by [18] for more practical in large-scale industrial settings. xDeepFM [10] considers both the implicit and explicit high-order feature interactions and devises CIN to model the feature interactions in an explicit manner and at a vectorwise level. The adaptive factorization network (AFN) [19] models the arbitrary-order cross features and their weights adaptively via a logarithmic transformation network. On the other hand, the attention mechanism is also introduced for multifield sparse data to improve the feature interactions by highlighting useful features. Attentional factorization machines (AFMs) [20] introduce the method of attention mechanism to learn the importance of each second-order feature interaction based on FM. High-order attentive FM (HoAFM) [21] aggregates the representations of other co-occurred features and learns the different importance of co-occurred features on the granularity of dimensions. AutoInt [11] employs a multihead self-attentive neural network with residual connections to model the different orders of feature interactions. FiBiNET [12] dynamically captures the significance of features via the SENET. Dual input-aware factorization machines (DIFMs) [22] adaptively reweight the original feature representations at the bitwise and vectorwise levels simultaneously. In these optimized methods, the amorphous combination or independent attention unit still lacks the competence of learning sophisticated interactions among different feature fields. Therefore, this article will focus on how to model the sophisticated high-order interactions effectively in a more flexible and explicit manner.

B. Graph Neural Networks

Graph structure consists of nodes and edges, where the edge represents the relationship of connected nodes. In the past decade, a growing body of research is interesting in graphs' representation, acquisition, and applications [23]. Concretely, Xiao *et al.* [24] characterize graphs to measure similarity and clustering by computing permutation invariants from the heat kernel trace. Han *et al.* [25] construct a generative prototype for graphs by adopting a minimum description length approach. Bai *et al.* [26] propose two novel local-global nested graph kernels, which can simultaneously reflect the local and global graph characteristics in terms of the nested complexity traces. Recently, a number of deep learning algorithms have been applied to irregular inputs, such as graph structure data. Due to the massively parallel architectures, modern graphics processing units (GPUs) have been successfully utilized

to accelerate the performance of graph computation [27]. Frascioni *et al.* [28] utilize recursive neural networks to represent and process data in graph. DeepWalk [29] uses truncated random walks to learn node latent representations in the graph. Following the DeepWalk, the large-scale information network embedding (LINE) algorithm [30] preserves both the local and global structural information to suit arbitrary types of the information embedding network. Later, Cavallari *et al.* [31] consider embedding communities instead of individual nodes, which can identify the interaction between community embedding and detection as a closed loop, through node embedding. However, most of these shallow models cannot learn nonlinear network structure, resulting in suboptimal representations for large graphs.

GNN [32] is designed to address the above deficiencies, which is a type of neural network that performs neural network operations via graph structure to acquire node representations. A typical GNN model consists of an iterative random-walk process in which node states are propagated by aggregating information from neighborhoods. Recently, numerous GNN variants with various types of novel aggregators and updaters have been proposed. For instance, GCN [15] tackles spectral information by employing the convolutional aggregator to operate the first-order neighborhood around each node. Graph sample and aggregate (GraphSAGE) [33] proposes three types of aggregators: mean aggregator, long short-term memory (LSTM) aggregator, and pooling aggregator. Graph attention networks (GATs) [34] capture different weights to different nodes in a neighborhood by stacking layers. Moreover, cross-GCN [35] introduces a cross-feature operator to explicitly model the arbitrary-order cross features with complexity linear to feature dimension and order size.

Recently, GNN has been broadly applied to various tasks, such as recommender system [1], action recognition [36], semantic segmentation [37], image recognition [38], and visual question answering [39]. Fi-GNN [13] first introduces GNN to model feature interactions on a graph structure. However, its simple aggregation manner limits the capability of cross features, and it also cannot learn the impact of aggregation among nodes. Thus our work attempts to deeply integrate GNN and attention mechanisms to model feature interactions of graph structure features for multifield sparse data.

C. Broad Learning System

Due to the deep structure and abundant connecting weights, most networks based on deep learning usually put up with a time-consuming training process. In order to resolve this shortcoming of the long training process problem in the deep network, Pao and Takefuji [40] design a random vector function-link neural network (RVFLNN). It transforms hidden nodes in the traditional single-layer feedforward neural network (SLFN) into enhancement nodes and connects the output node with both input and enhancement nodes. Besides, the parameters transforming input nodes to enhancement nodes are randomly generated. This single change has greatly improved the performance of this network.

For the sake of resolving the high dimensional data problem and long training process, BLS [16] is proposed as a substitute for deep structure, which is inspired by RVFLNN and the dynamic stepwise updating algorithm [41]. BLS is designed as a flat network, where the original feature nodes are transferred as “mapped features,” and the structure is expanded in a wide sense in the “enhancement nodes.” Moreover, the connection

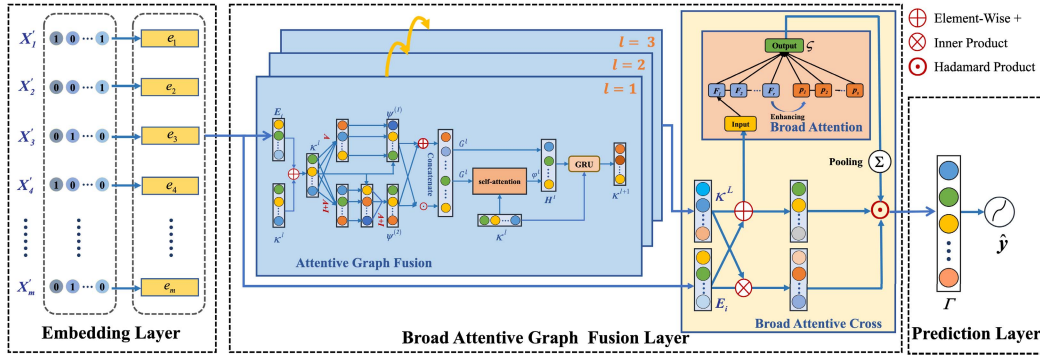


Fig. 1. Overall architecture of our BaGFN model. The new model consists of three layers: embedding layer, broad attentive graph fusion layer, and prediction layer. As the core of the BaGFN model, the second layer consists of two key parts: the attentive graph fusion module and the broad attentive cross module. The first module strengthens high-order feature representation by integrating graph structure and self-attention mechanism. The second module designs a broad attention mechanism to refine high-order feature interactions at the bitwise level.

weights are updated using ridge regression of the pseudoinverse method. Compared with deep learning methods, BLS can be easily constructed at a fast speed, even without a high-performance computer. In addition, due to the independence and flexibility of the “mapped features” and “enhancement nodes,” there are many advanced variants of BLS [42] to improve the performance. In this article, we introduce the BLS theory to construct a new broad attention module, which can achieve the efficient refinement of high-order feature interactions at a bitwise level.

III. BROAD ATTENTIVE GRAPH FUSION NETWORK

In this section, we will introduce the overview of our BaGFN model, which is illustrated in Fig. 1. The new architecture is composed of three components.

- 1) *Embedding layer* that converts the raw features of multifield sparse data into the embedding vectors.
- 2) *Broad attentive graph fusion layer* that is the core of our model. It contains two parts: attentive graph fusion module and broad attentive cross module. The first module integrates graph structure and self-attention mechanism to refresh high-order features representation. The second module further refines high-order feature interactions by designing a broad attention mechanism at the bitwise level.
- 3) *Prediction layer* that combines the above cross features and then computes the final prediction probability. The mathematical notations used in this article are summarized in Nomenclature.

A. Embedding Layer

In the multifield sparse data, each input instance is made up of several field-aware values, which includes plenty of dynamic numerical fields and static categorical fields. For example, a movie instance has some categorical fields: {Language: Italian, Region: Italy, Director: Roberto Benigni, ...}, which cannot be directly entered into a DNN-based model. Therefore, we should convert the instance into multifield representations, including numerical representations from numerical fields and one-hot representations from categorical fields

$$X_i = [X'_1, \dots, X'_m] = [\underbrace{1, 0, \dots, 0}_{\text{field}_1}, \dots, \underbrace{0, 1, \dots, 0}_{\text{field}_m}] \quad (1)$$

where X'_m is one-hot encoding representation in the m th field; m is the size of fields; and X_i represents the i th input

instance with m field representations. Following the former works [10], [17], [21], we apply the embedding layer to transform each field representation into the low-dimensional and dense embedding vector. Finally, the embedding output $E_i = [e_1, e_2, \dots, e_p, \dots, e_m]$ represents m embedding vectors of one input instance X_i , where $e_p \in \mathbb{R}^d$ denotes the embedding vector of the p th field and d is the dimension of embedding layer.

B. Broad Attentive Graph Fusion Layer

1) *Attentive Graph Fusion Module*: In this module, the embedding instance E_i is defined as multifield input features. Next, we express these features in the form of a graph, that is, $G = (N, \xi)$. Each node $n_p \in N$ in the graph represents a feature field p with embedding vector e_p , and the number of nodes $|N|$ is m . The edge ξ reflects intricate interactions between feature fields, and the edge weights denote the importance of different feature interactions. After the definition of graph structure, the module will optimize the adjacency matrix of edge weights, design a new bilinear-cross aggregation function, learn the aggregation impact of neighborhood nodes, and construct multihop feature fusion steps. These operations can effectively strengthen high-order feature representation and, finally, yield higher quality cross features. Concretely, each node must qualify the impact of its neighbors and aggregate their cross information to output high-quality feature fusion of sophisticated node interactions. On the other hand, the steps of feature fusion can be further connected to generate high-order feature representation, called multihop feature fusion steps, i.e., the input of the latter step comes from the output of the former step. Formally, $H^l = f(\kappa^l, A, W^l)$ represents the node interactions representation results at feature fusion step l , where $f(\cdot)$ is the aggregation function; $A \in \mathbb{R}^{m \times m}$ is the adjacency matrix with edge weights, where m is the number of nodes; κ^l is the input features at step l , which needs to execute the elementwise addition with multifield input features E_i ; and W^l denotes the trainable matrix of network weights. Then, H^l and κ^l are entered into a gate recurrent unit (GRU), which can generate the input features κ^{l+1} at the next step $l+1$. Finally, this module can yield the final result κ^L of feature representation, where L is the number of multihop feature fusion steps. Apparently, the adjacency matrix A and the aggregation function $f(\cdot)$ are fundamental to the effectiveness of node interactions in this module.

a) *Attentional Edge Weights*: Generally, different graph-based models deploy a variety of strategies to

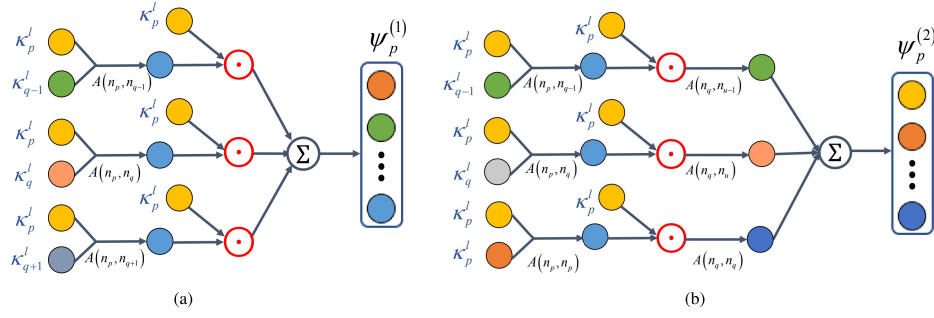


Fig. 2. Architectures of different aggregation operations. In order to fully extract the cross features, a new bilinear-cross aggregation function integrates two different aggregation operations. (a) First-order aggregation. (b) Second-order aggregation. The former is to enhance the affinity between the current node and its neighborhood nodes. The latter emphasizes building a bridge between two nodes without a direct connection.

construct the adjacency matrix of edge weights. For instance, the conventional GNN model [32] usually adopts 0 and 1 to reflect the relationship between nodes. This way is too simple and rough to generate the accurate adjacency matrix for sophisticated node interactions. Thus, Fi-GNN [13] utilizes the attention mechanism to explicitly learn the edge weights as follows:

$$\zeta(n_p, n_q) = \frac{\exp(\text{LeakyReLU}(W_\zeta[e_p \| e_q]))}{\sum_{k \in \mathcal{N}_p} \exp(\text{LeakyReLU}(W_\zeta[e_p \| e_k]))} \quad (2)$$

where $\zeta(n_p, n_q)$ denotes the edge weight between n_p and n_q ; e_p and e_q are two embedding vectors of n_p and n_q ; $\mathcal{N}_p$ is the neighborhood nodes of n_p ; $\|$ is the concatenation operation; W_ζ is the trainable matrix of attentive weights; and the leaky rectified linear unit (LeakyReLU) is the nonsaturated activation function. Recently, some state-of-the-art research work [6], [17] has proved that the product operation is better than the concatenate operation for multifield feature interactions. Therefore, different from Fi-GNN [13], we propose a more fine-grained method to represent the edge weights as follows:

$$\zeta(n_p, n_q) = \frac{\exp(\text{ReLU}(\varrho \odot (\tilde{e}_p \odot \tilde{e}_q)))}{\sum_{k \in \mathcal{N}_p} \exp(\text{ReLU}(\varrho \odot (\tilde{e}_p \odot \tilde{e}_k)))} \quad (3)$$

where $\odot$ denotes the Hadamard product operation that is elementwise multiply, for example, $[c_1, c_2, c_3] \odot [d_1, d_2, d_3] = [c_1d_1, c_2d_2, c_3d_3]$; ϱ is the trainable vector of attentive weights; $\tilde{e}_p$ and $\tilde{e}_q$ are the normalized vectors of e_p and e_q ; and the rectified linear unit (ReLU) is a new activation function, which is used to enhance the sparsity of feature selection instead of LeakyReLU. Accordingly, the adjacency matrix A can be defined as

$$A(n_p, n_q) = \begin{cases} \zeta(n_p, n_q), & q \neq p \\ 0, & \text{otherwise} \end{cases} \quad (4)$$

where $A(n_p, n_q)$ denotes the edge weight from node n_p to its neighborhood node n_q . In brief, we construct a more fine-grained representation of attentional edge weights to accurately reflect the importance of node interactions.

b) Bilinear-Cross Aggregation: With the adjacency matrix A and the input features κ^l , the aggregation function $f(\cdot)$ will further update the state of each node by aggregating the cross information between itself and its neighborhood nodes. Generally, the traditional aggregation operation in GNN [32] is the simple multiply of adjacency matrix and node information, which is difficult to fully extract the cross features. Inspired by some graph-based research studies [43], [44], we propose a new bilinear-cross aggregation

function, which integrates two different aggregation operations: first-order aggregation and second-order aggregation. The former is to enhance the affinity between the current node and its neighborhood nodes. The latter emphasizes building a bridge between two nodes without a direct connection. As shown in Fig. 2(a), the first-order aggregation operation $\psi_p^{(1)}$ can update the state of the current node n_p by connecting with its first-order neighborhood nodes $\mathcal{N}_p^{(1)}$, which is defined as

$$\psi_p^{(1)} = \sum_{n_q \in \mathcal{N}_p^{(1)}} A(n_p, n_q) \kappa_p^l \odot \kappa_q^l \quad (5)$$

where $A(n_p, n_q)$ denotes the edge weight between n_p and n_q ; κ_p^l denotes the input features of n_p at step l . $\odot$ denotes the Hadamard product operation. Besides the multiply of the adjacency matrix and input features, the first-order aggregation adds the product operation with the input features, which is beneficial to enhance the affinity between nodes. As shown in Fig. 2(b), unlike the first-order aggregation, the second-order aggregation operation $\psi_p^{(2)}$ considers the current node n_p to connect its second-order neighborhood nodes $\mathcal{N}_p^{(2)}$, which is defined as

$$\psi_p^{(2)} = \sum_{n_u \in \mathcal{N}_p^{(2)}} (A(n_q, n_u) + I)((A(n_p, n_q) + I) \kappa_p^l \odot \kappa_q^l) \quad (6)$$

where $n_u \in \mathcal{N}_p^{(2)}$ denotes the second-order neighborhood node of n_p , i.e., $n_u \in \mathcal{N}_q^{(1)}$, $n_q \in \mathcal{N}_p^{(1)}$; $A(n_q, n_u)$ denote the edge weight between n_q and n_u ; and I denotes the unit matrix. The second-order aggregation not only adopts two-hop propagation to enlarge the aggregation range but also reflects the interaction effect of the current node itself using the unit matrix. Finally, we integrate the first- and second-order aggregations into a new bilinear-cross aggregation function to improve the representation of node interactions with different cross orders as follows:

$$G_p^l = W_\psi^l [\psi_p^{(1)} \oplus \psi_p^{(2)}, \psi_p^{(1)} \odot \psi_p^{(2)}] \quad (7)$$

where G_p^l denotes the bilinear-cross aggregation result for n_p ; W_ψ^l denotes the trainable matrix of node aggregation weights; and $\oplus$ and $\odot$ are the elementwise addition and the Hadamard product, respectively.

Meanwhile, we apply the self-attention mechanism to get the aggregation impact from the neighborhood nodes to the current node. Thus, we can define the attention weight vector ϕ_p^l of n_p to generate the representation result $H_p^l = G_p^l \odot \phi_p^l$

of node interactions at the feature fusion step l , which can be calculated as follows:

$$\phi_p^l = \frac{\exp(W_2^l \text{ReLu}(W_1^l [G_p^l \| k_p^l]) \odot k_p^l)}{\sum_{k \in \mathcal{N}_p} \exp(W_2^l \text{ReLu}(W_1^l [G_k^l \| k_k^l]) \odot k_k^l)} \quad (8)$$

where G_p^l and G_k^l denote the bilinear-cross aggregation results for n_p and its neighbor n_k , respectively; W_1^l and W_2^l denote two trainable matrices of node attention weights; k_p^l denotes the input features of n_p at step l ; $\mathcal{N}_p$ is the neighborhood nodes of n_p ; $\|$ is the concatenation operation; $\odot$ is the Hadamard product; and ReLu is the activation function.

Finally, we introduce the GRU module to obtain the updated representation k_p^{l+1} of n_p at the next step $l+1$, which is calculated as follows:

$$\begin{aligned} z_p^{l+1} &= \text{sigmoid}(W_z^l H_p^l + U_z^l \kappa_p^l + b_z^l) \\ r_p^{l+1} &= \text{sigmoid}(W_r^l H_p^l + U_r^l \kappa_p^l + b_r^l) \\ \tilde{\kappa}_p^{l+1} &= \tanh(W_h^l H_p^l + U_h^l (r_p^{l+1} \odot \kappa_p^l) + b_h^l) \\ \kappa_p^{l+1} &= \tilde{\kappa}_p^{l+1} \odot z_p^{l+1} + H_p^l \odot (1 - z_p^{l+1}) \end{aligned} \quad (9)$$

where W_z^l , U_z^l , b_z^l , W_r^l , U_r^l , b_r^l , W_h^l , U_h^l , and b_h^l are the trainable parameters of the GRU module; sigmoid and tanh are two activation functions. According to a series of operations for n_p , we can update the representations of all nodes to yield the input features κ^{l+1} at the next step $l+1$. After L propagation steps, we will output the final result κ^L of high-order feature representation in this module.

2) *Broad Attentive Cross Module*: The attentive graph fusion module learns the node interactions at the vectorwise level where all feature representations of cross nodes share the same weights, which cannot specify the significance of each interaction at the granularity of feature dimensions. Considering this drawback, we propose a broad attentive cross module to efficiently refine high-order feature interactions at the bitwise level. The new module is designed to dynamically increase the weights of important nodes and decrease the weights of uninformative nodes.

Generally, feature nodes are generated independently and have no relationship with each other by feature mapping in traditional BLS [16]. Unfortunately, the separated feature mapping will reduce the intrinsic correlation of input data. In this article, we consider that modeling feature interactions is to capture cross features among multifield data with complex intrinsic correlation. Therefore, inspired by time-delayed neural networks [42], [45]–[47], we reconstruct a cascade BLS in the form of recurrent, which can strengthen the correlation between nodes. As shown in Fig. 3, we design a new broad attention mechanism based on the idea of cascading to construct the feature nodes and the enhanced nodes, strengthening the correlation between nodes to improve the performance of feature mapping. The cascaded feature mapping can be defined as follows:

$$F_{z_i} = \phi^{z_i}(\phi^{z_i-1}(F_{z_{i-1}} W_{e_{j-1}} + \beta_{e_{j-1}})) W_{e_j} + \beta_{e_j} \quad (10)$$

where F_{z_i} is the z_i th mapping feature nodes, $F_0 = \kappa^L \oplus E_i$, $z_i \in [1, r]$, r is the number of feature mapping; W_{e_j} and β_{e_j} are randomly generated weights and bias; $\oplus$ denotes the elementwise addition; and $\phi^{z_i}(\cdot)$ denotes the LeakyReLu activation function. All mapping features are concatenated to generate the final feature nodes $F^r \triangleq [F_1, \dots, F_r]$. Next,

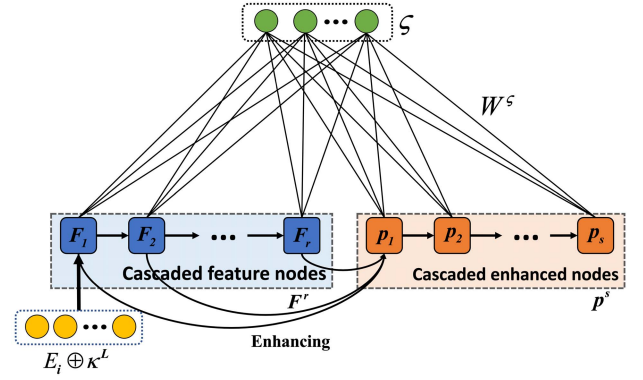


Fig. 3. Architecture of broad attention. The broad attention mechanism can construct the cascaded feature mapping to generate the feature nodes and the enhanced nodes and then calculate the attentive weights of global-aware nodes to reflect the importance of each cross feature.

we execute the enhanced feature mapping as follows:

$$P_{f_i} = \phi^{f_i}(\phi^{f_i-1}(P_{f_{i-1}} W_{h_{j-1}} + \beta_{h_{j-1}})) W_{h_j} + \beta_{h_j} \quad (11)$$

where P_{f_i} is the f_i th enhanced feature node; $P_0 = F^r$; $f_i \in [1, s]$; s is the number of enhanced feature mapping; W_{h_j} and β_{h_j} are randomly generated weights and bias; and $\phi^{f_i}(\cdot)$ denotes the LeakyReLu activation function. All enhanced mapping features are concatenated to generate the final enhanced nodes $P^s \triangleq [P_1, \dots, P_s]$.

After obtaining the feature nodes F^r and the enhanced nodes P^s , we can calculate the broad attentive weights ζ of global-aware nodes, reflecting the importance of each cross feature

$$\begin{aligned} \zeta &= [F^r \| P^s] W^\zeta \\ W^\zeta &= [F^r \| P^s]^+ \zeta = A^+ \zeta \\ A^+ &= \lim_{\lambda \rightarrow 0} (\lambda I + A^T A)^{-1} A^T \end{aligned} \quad (12)$$

where $\|$ is the concatenation operation; W^ζ is the trainable matrix of connecting weights; A^+ is the pseudoinverse of $A = [F^r \| P^s]$; λ is an adjustment parameter with an initial value of 0.1; and I is the unit matrix.

Finally, the representation Γ of cross features can be calculated as follows:

$$\Gamma = (W_2^b(\kappa^L \otimes E_i)) \odot (W_1^b(\kappa^L \oplus E_i)) \odot \sigma(\text{pooling}(\zeta)) \quad (13)$$

where W_1^b and W_2^b are two trainable matrices of network weights; ζ denotes the broad attentive weights of global-aware nodes; $\sigma(\cdot)$ denotes the sigmoid activation function; $\odot$ denotes the Hadamard product; $\otimes$ denotes the inner product; and $\oplus$ denotes the elementwise addition. In this module, the broad attentive weights reflect the significance of node interactions. The high-order feature interactions are conducted at the bitwise level, which can achieve a more fine-grained representation at each dimension of cross features.

C. Prediction Layer

After obtaining the high-quality cross features by the above layer, we can compute the final predictive result of our BaGFN model for multifield sparse data, which is defined as follows:

$$\hat{y} = \text{sigmoid}\left(w + \sum_{i=1}^{|\Gamma|} \Gamma_i\right) \quad (14)$$

where $\hat{y} \in [0, 1]$ is the final prediction probability; sigmoid is an activation function; Γ is the final representation of feature interactions; w is a bias parameter; and $|\Gamma|$ is the dimension of cross features Γ .

In the process of training, our BaGFN model takes log-likelihood loss as the loss function and then minimizes the objective function $\mathcal{L}$, which is defined as

$$\mathcal{L} = -\frac{1}{M} \sum_{i=1}^M (y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)) \quad (15)$$

where M is the size of the training samples; y_i is the predictive ground truth of the i th training instance; and $\hat{y}_i$ is the predictive result of our new network. Meanwhile, we apply backpropagation to continually update the trainable weight matrices of the above layers until the minimum value of the loss is yielded. The overall training process of our BaGFN model is illustrated in Algorithm 1.

D. Complexity Analysis

Referring to the computation mode of [11], the time consumption of computing an instance in Algorithm 1 mainly comes from four operations: calculating embedding output, constructing adjacency matrix, iteratively updating graph nodes, and computing broad attention. First, since the one-hot encoding representation of each field is transformed into a fixed low-dimensional embedding vector, the time complexity of calculating embedding output is represented as $O(Cd)$, where C is the total length of all one-hot encoding representations and d is the dimension of embedding space $\mathbb{R}^d$. Second, the time complexity of constructing an $m \times m$ adjacency matrix is $O(m^2d)$, where m is the number of fields and also the number of graph nodes. Third, when updating a graph node, the time complexity of bilinear-cross aggregation, including the first-order aggregation and the second-order aggregation, is defined as $O(md + m^2d)$. Since the self-attention and GRU operations will transform each other between $\mathbb{R}^d$ and a new space $\mathbb{R}^{d'}$, their time complexity is represented as $O(mdd')$. Thus, when iteratively updating all graph nodes, the time complexity can be computed as $O(Lm(md + m^2d + mdd'))$, where L is the number of propagation steps. Fourth, since the broad attention module needs to carry out the cross calculation between nodes, its time complexity is expressed as $O(mn^2d^2)$, where n is the number of nodes in cascade BLS. Finally, the time complexity of computing an instance is defined as $O(Cd + m^2d + Lm^2d + Lm^3d + Lm^2dd' + mn^2d^2)$, which can be simplified to $O(C + m^2(dd' + md) + mn^2d^2)$ because d and L are usually small, and C is usually large and $C \gg d$. When training all instances in parallel, the final time complexity of our proposed method is $O(M'(C + m^2(dd' + md) + mn^2d^2))$, $M' = M \times \text{IN}/\text{bsize}$, where M is the size of training instances, IN is the number of training iteration, and bsize is the size of a batch.

According to the definition of [13], the space complexity of our proposed method is a measure of the total amount of parameters, which consists of trainable matrices of several key parts, including the embedding layer, the attentive graph fusion module, and the broad attentive cross module. First, the average size of embedding matrices is defined as $c_1 \times d$, where c_1 is the average length of all one-hot encoding representations. Thus, the space complexity of the embedding layer is $O(mc_1d)$. Second, since the attentive graph fusion

Algorithm 1 Training of Our BaGFN Model

Input: All training samples $(X_i, y_i), i \in [1, M]$, X_i and y_i are the encoding input and the predictive ground truth of the i th training instance, respectively, M is the size of the training samples

Output: The trained network weights W

```

1: Set the batch size bsize and the iteration number IN;
2: Initialize all network weights  $W_0$ ;
3: for  $t = 1; t \leq IN$  do
4:   Execute a batch of training instances in parallel.
5:   Obtain the embedding output  $E_i$  from  $X_i$ ;
6:   Construct the adjacency matrix  $A$  from  $E_i$ ;
7:   for  $l \leftarrow 1$  to  $L$  do
8:     for each node  $p$  do
9:       Compute the bilinear-cross aggregation  $G_p^l$ ;
10:      Compute the self-attention weight vector  $\phi_p^l$ ;
11:      Get the representation  $H_p^l$  from  $(G_p^l, \phi_p^l)$ ;
12:      Use the GRU module to obtain  $\kappa_p^{l+1}$ ;
13:     end for
14:     Generate  $\kappa^{l+1}$  after updating all nodes;
15:   end for
16:   Generate  $\kappa^L$  after  $L$  propagation steps;
17:   for  $z_i \leftarrow 1$  to  $r$  do
18:     Calculate each feature node  $F_{z_i}$  from  $(\kappa^L, E_i)$ ;
19:   end for
20:   Obtain all feature nodes  $F^r$ ;
21:   for  $f_i \leftarrow 1$  to  $s$  do
22:     Calculate each enhanced node  $P_{f_i}$  from  $F^r$ ;
23:   end for
24:   Obtain all enhanced nodes  $P^s$ ;
25:   Construct the broad attentive weights  $\zeta$  from  $(F^r, P^s)$ ;
26:   Calculate the cross features  $\Gamma$  from  $(\kappa^L, E_i, \zeta)$ ;
27:   Obtain the final predictive result  $\hat{y}$  from  $\Gamma$ ;
28:   Minimize the objective function  $\mathcal{L}$  from  $(\hat{y}, y)$ ;
29:   Update  $W_t$  using Back Propagation for  $\mathcal{L} \downarrow$ ;
30: end for
31: return  $W \leftarrow W_{IN}$ ;

```

module will transform each other between $\mathbb{R}^d$ and a new space $\mathbb{R}^{d'}$, the size of each trainable matrix can be defined as $d \times d'$. In an iteration step, the size of all trainable matrices is $c_2 \times d \times d'$, where c_2 is the number of matrices. Thus, the space complexity of the attentive graph fusion module can be defined as $O(Lc_2dd')$. Third, the size of the trainable broad attention matrix is $c_3d \times c_3d$, where c_3 is the number of nodes in cascade BLS. Thus, the space complexity of the broad attentive cross module is expressed as $O(c_3^2d^2)$. Finally, since the parameters of these trainable matrices are shared, the final space complexity is defined as $O(c_1md + c_2Ldd' + c_3^2d^2)$, which is fixed during the training of our BaGFN model.

IV. EXPERIMENTAL RESULTS

In this section, we conduct extensive experiments to answer the following questions.

- 1) *RQ 1*: How does our model perform in learning cross features compared with the state-of-the-art models on multifield sparse data? Is it efficient for model training?

TABLE I
STATISTICS OF THREE EVALUATION DATASETS

Dataset	Instances	Fields	Features (Sparse)
Avazu	40,428,967	23	1,544,488
Criteo	45,840,617	39	998,960
Tobacco	166,626	249	146,103

- 2) *RQ 2*: How do the different combinations of aggregation function types (first-order, second-order, bilinear-cross, and bilinear-concatenate) and edge weights (concatenate and product operations) in attentive graph fusion module impact its performance?
- 3) *RQ 3*: How do the different settings of the network influence the performance of our model? Can our model reflect the significance of different feature fields?
- 4) *RQ 4*: Similar to the state-of-the-art models, does our model also further enhance the performance by integrating implicit feature interactions?

We will deal with these questions after presenting several fundamental experimental configurations.

A. Experimental Configuration

1) *Datasets*: In order to evaluate the validity of our BaGFN model, we conduct a series of experiments on two well-known public benchmark datasets and a new industrial dataset. The statistics of the datasets are summarized in Table I.

a) *Avazu*: The Avazu dataset consists of users' click records of several days on mobile advertisements with 40 million data instances. This dataset is initially used for Kaggle CTR prediction competition and later widely used in the evaluation benchmark for many CTR prediction models. For each click instance, it has 23 fields spanning from user/device features to add attributes.

b) *Criteo*: Criteo is a famous real-world display ad dataset with each ad display information and corresponding user click logs. This dataset is widely used in the evaluation of various CTR prediction models. It contains users' click logs with 45 million samples, and each sample has 39 fields, including 13 continuous numerical fields and 26 categorical fields.

c) *Tobacco*: The new dataset consists of more than 160000 inspection records for 40000 tobacco stores from 2014 to 2018. Similar to CTR prediction, this dataset is to predict illegal sales in the real-world tobacco industry. Each sample contains 249 fields (195 dynamic numerical fields and 54 static categorical fields) from individual information, dynamic orders, and sales log and has numerous null values and a few anomaly data.

d) *Data preparation*: For Avazu and Criteo datasets, we remove the infrequent values appearing less than ten and five times, respectively, and treat them as a single value (unknown). Since the numerical fields may have large variance and then hurt the model performance, we normalize numerical values by transforming a value z to $\log_2(z)$ if $z > 2$, which is first proposed by the winner of Criteo Competition.³ For the Tobacco dataset, we replace the null value with -1 and normalize numerical values to eliminate the quantum influence between different ranges of numerical fields. For

three datasets, we randomly select 80% of all samples for training and randomly divide the rest into equal validation sets and test sets. In this article, all methods follow this unified way of data preparation for a fair comparison.

2) *Evaluation Metrics*: Thus, to evaluate the prediction performance of our BaGFN model on Avazu, Criteo, and Tobacco datasets, we adopt two popular metrics.

Area Under the ROC Curve (AUC) measures the probability that a predictive model will allocate a higher score to a positive instance than a randomly chosen negative instance. AUC's upper bound is 1, and the larger value indicates a better performance.

Logloss (cross entropy) is the cross-entropy loss to assess the performance of a classification model, measuring the distance between the predicted score and the true label for each sample. The smaller Logloss value denotes better performance.

3) *Comparison Methods*: We compare our BaGFN model with three types of existing methods: 1) modeling first-order interactions that linearly sum up raw features; 2) FM-based methods that learn second-order cross features; and 3) deep learning-based models that model high-order feature interactions. We briefly describe these methods as follows.

- 1) *Logistic regression (LR)* [48] captures first-order cross features via summing up raw individual features linearly.
- 2) *FM* [6] is a benchmark factorization model, which models the second-order interactions between paired features from the inner product of vectors.
- 3) *AFM* [20] is an extension of FM, which learns the significance of different second-order cross features with the help of the attention mechanism.
- 4) *CrossNet (Deep&Cross)* [2] attempts to model feature interactions in an explicit manner by taking the outer product of feature vectors at the bitwise level.
- 5) *DeepCrossing* [49] integrates the advantages of deep fully-connected neural networks and residual connections to capture nonlinear high-order feature interactions via an implicit manner.
- 6) *NFM* [8] feeds the elementwise product of feature vectors into a bi-interaction pooling layer to model the second-order interactions and then adopts deep fully-connected neural networks to capture nonlinear high-order interactions in an implicit fashion.
- 7) *PNN* [17] concatenates pairwise inner or outer products of input feature vectors to learn high-order feature interactions.
- 8) *CIN* is the core of xDeepFM [10], which produces high-order cross features by computing the outer product of input feature vectors and then compressing the feature maps originated from the outer product to renew the representation of each feature vector.
- 9) *High-order FM (HOFM)* [50] is a high-order version of FM, which proposes efficient kernel-based algorithms for modeling nonlinear high-order factorization machines.
- 10) *AutoInt* [11] utilizes a multihead self-attentive neural network with residual connections to model the high-order feature interactions in an explicit manner.
- 11) *HoAFM* [21] designs a cross interaction layer and bitwise attention mechanism to account for the high-order sparse feature interactions in an explicit manner.
- 12) *Fi-GNN* [13] generates sophisticated feature interactions by designing a novel GNN to model the graph-structured features in an explicit manner.

³<https://www.csie.ntu.edu.tw/~r01922136/kaggle-2014-criteo.pdf>

TABLE II
OVERALL PERFORMANCES OF VARIOUS MODELS ON AVAZU, CRITEO, AND TOBACCO DATASETS

Model Category	Model	AUC	Avazu Logloss	p-value	AUC	Criteo Logloss	p-value	AUC	Tobacco Logloss	p-value
First-Order	LR [48]	0.7558	0.3963	0.00031	0.7822	0.4696	0.00022	0.7243	0.4493	0.00019
Second-Order	FM [6]	0.7710	0.3855	0.00058	0.7833	0.4695	0.00085	0.7883	0.4111	0.00011
	AFM [20]	0.7716	0.3852	0.00067	0.7941	0.4581	0.00047	0.7958	0.4092	0.00019
High-Order	NFM [8]	0.7711	0.3863	0.00018	0.7959	0.4563	0.00062	0.7926	0.4083	0.00059
	DeepCrossing [49]	0.7648	0.3886	0.00075	0.8010	0.4512	0.00022	0.8698	0.3561	0.00063
	CrossNet [2]	0.7671	0.3867	0.00021	0.7908	0.4589	0.00011	0.8618	0.3588	0.00082
	PNN [17]	0.7753	0.3835	0.00028	0.7998	0.4521	0.00010	0.8736	0.3461	0.00051
	HOFM [50]	0.7703	0.3853	0.00011	0.8008	0.4506	0.00071	0.8755	0.3473	0.00034
	CIN [10]	0.7759	0.3829	0.00045	0.8011	0.4515	0.00063	0.8801	0.3398	0.00071
	AutoInt [11]	0.7755	0.3822	0.00099	0.8063	0.4453	0.00061	0.8369	0.3951	0.00069
	HoAFM [21]	0.7761	0.3820	0.00090	0.8057	0.4468	0.00031	0.8839	0.3395	0.00059
	Fi-GNN [13]	0.7762	0.3825	0.00035	0.8062	0.4453	0.00077	0.8843	0.3353	0.00065
	DCN-M* [18]	0.7649	0.3880	0.00068	0.7992	0.4514	0.00038	0.8820	0.3465	0.00039
	DIFM* [22]	0.7660	0.3874	0.00019	0.7905	0.4592	0.00059	0.8702	0.3544	0.00089
	BaGFN	0.7803	0.3794	—	0.8094	0.4425	—	0.8870	0.3329	—

13) *DCN-M** is an updated version of DCN-M [18] without implicit feature interactions, which improves the ability of CrossNet [2] to make it more practical in large-scale industrial settings.

14) *DIFM** is an updated version of DIFM [22] without implicit feature interactions, which integrates various components and reweights the original feature representations at the bitwise and vectorwise levels simultaneously.

4) *Implementation Details*: In this article, all experiments are conducted over a server equipped with two NVIDIA Titan XP GPUs. Here, we refer to the default or optimal parameter configuration of these state-of-the-art models. For DeepCrossing, we employ four feedforward layers, each with 100 hidden units. For NFM, we adopt one hidden layer with size 200 on top of the bi-interaction layer. For HoAFM, we set its depth to three. For CrossNet and CIN, we employ three interaction layers. For our BaGFN model, we mainly consider the configuration of five parameters, including embedding dimension, epochs, iteration steps, batch size, and learning rate. For three different evaluation datasets, we conduct a series of comparison experiments to find the best parameters settings and yield the optimal prediction results. Moreover, we apply the Adam optimizer to optimize all DNN-based models.

B. Performance Comparison (RQ1)

According to the above experimental configurations, we compare our BaGFN model with all kinds of individual models, including first-, second-, and high-order feature interactions. As illustrated in Table II and Fig. 4, we summarize the performance of these models and obtain the following observations.

- 1) The linear model LR fails to yield a good performance on the three datasets compared to the other nonlinear models, suggesting that modeling nonlinear relations among features is essential to improve the accuracy of CTR prediction.
- 2) FM and AFM, which capture second-order nonlinear feature interactions, achieve better predictive results compared to LR, showing that the pairwise interactions via the inner product of feature vectors have a positive impact on prediction performance. Moreover, AFM generates better predictive results than FM on

all three datasets, which indicates that the attention on different feature interactions endows the model with better interaction ability.

- 3) Most high-order models substantially outperform those of learning low-order feature interactions on the three evaluation datasets. This is consistent with the intuition that modeling high-order interactions among features is more significant for prediction performance.
- 4) Fi-GNN exhibits better prediction performance compared to the other baseline models, indicating that feature interactions based on graph neural networks have the stronger capability of feature representation.
- 5) Our BaGFN model yields the best performance on all three datasets and achieves innovative improvements over the state-of-the-art models. BaGFN has the highest AUC values (0.7803, 0.8094, and 0.8870) and the lowest Logloss values (0.3794, 0.4425, and 0.3329) on Avazu, Criteo, and Tobacco. The previous works [2], [9], [11], [13] have proved that the improvement with respect to AUC at 0.001-level is significant for the CTR prediction task. Our proposed model presents great superiority over those state-of-the-art models due to the effectiveness of integrating graph structure and attention mechanism on modeling feature interactions.
- 6) Compared with the other baseline models, the relative performance improvement of our BaGFN model is progressively higher on Avazu, Criteo, and Tobacco datasets, which may be because the number of feature fields is different across the three evaluation datasets, with the Tobacco dataset owning the most feature fields and the Avazu dataset owning the least. Generally, more feature fields are beneficial to capture more various cross features and achieve better performance improvement on the multifield sparse dataset.
- 7) For formal statistical analysis, the Wilcoxon test is introduced to calculate and compare the p-value, which indicates whether the differences in performance are statistically significant or not. With the prediction results of these models on Avazu, Criteo, and Tobacco datasets, we can calculate the p-values between our BaGFN model and the other models and further compare these p-values with the significance level to analyze their statistically significant differences. As illustrated in Table II, all p-values on three evaluation datasets

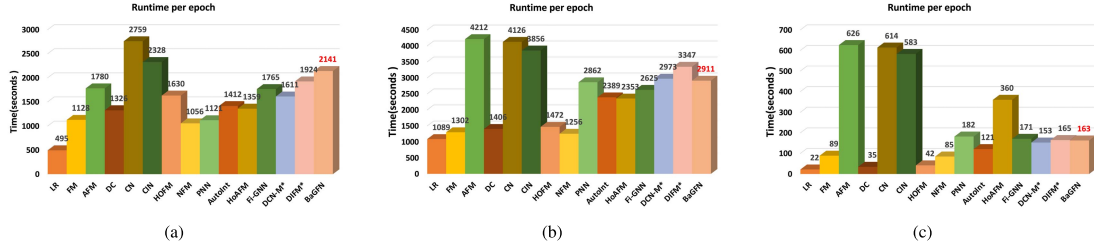


Fig. 4. Training runtimes of all models for each epoch on (a) Avazu, (b) Criteo, and (c) Tobacco datasets. Note that the baseline models include LR [48], FM [6], and AFM [20], DC represents DeepCrossing [49], CN represents CrossNet (Deep&Cross) [2], and CIN is the core of xDeepFM [10], HOFM [50], NFM [8], PNN [17], AutoInt [11], HoAFM [21], Fi-GNN [13], DCN-M* [18], and DIFM* [22]. Compared with the other models, our BaGFN model has a moderate time consumption for training.

are less than 0.01, which demonstrates that there is an extremely significant difference between our BaGFN model and the other state-of-the-art model.

- 8) Besides, we also analyze the running time in the process of model training. It is clear that LR becomes the most efficient model because of its simplicity. AFM, CrossNet, and CIN are the highest time-consuming models due to their complex architectures. The time complexity of our proposed method is $O(M'(C + m^2(dd' + md) + mn^2d^2))$, which indicates that the running time of training is related to the number of training instances, the total length of all one-hot representations, the number of graph nodes, and the embedding dimension. Since the number of fields and the embedding dimension are usually small, our BaGFN model will not add more training time than the other high-order models. Fig. 4 illustrates the time consumption of model training for one epoch on Avazu, Criteo, and Tobacco datasets. Compared with the other models, our BaGFN model has moderate time consumption for training.

C. Comparison of Different Aggregation Functions and Edge Weights (RQ2)

In this section, we exploit several variants of our model to investigate how the different aggregation functions and edge weights in the attentive graph fusion module affect the prediction results. For different aggregation functions, we utilize the original aggregation as same as GCN [15], termed $\text{model}_{\text{org}}$, and also construct two variants only using the first- and second-order operations in (5) and (6), which are termed $\text{model}_{\text{first}}$ and $\text{model}_{\text{second}}$, respectively. In addition, we define $\text{model}_{\text{concat}}$ to represent the concatenation operation of (5) and (6) and denote the bilinear-cross aggregation in (7) as $\text{model}_{\text{final}}$. Meanwhile, for computing edge weights, model_{ec} represents the concatenation operation as same as Fi-GNN [13], and $\text{model}_{\text{final}}$ indicates the product operation in (3). The final experimental results are shown in Table III with the following findings.

- 1) $\text{model}_{\text{first}}$ outperforms $\text{model}_{\text{org}}$ on three evaluation datasets, which indicates that the first-order operation can execute more effective node interactions than the original sum operation.
- 2) $\text{model}_{\text{second}}$ is not always better than $\text{model}_{\text{first}}$ and $\text{model}_{\text{org}}$, which indicates that the single second-order operation is unstable and needs to further combine the first-order operation for the stronger performance.
- 3) Unlike $\text{model}_{\text{first}}$ and $\text{model}_{\text{second}}$, $\text{model}_{\text{concat}}$ obtains the better performance on three evaluation datasets, which shows that concatenation operation can further improve

the ability of linear aggregation by integrating first- and second-order operations.

- 4) $\text{model}_{\text{final}}$ is superior to all types of aggregation operations, indicating that the bilinear-cross aggregation in our BaGFN model can significantly boost the prediction performance.
- 5) Compared with model_{ec} , $\text{model}_{\text{final}}$ achieves a significant improvement on three evaluation datasets, which indicates that the product operation in our BaGFN model can effectively compute the edge weights and better represent the relationship between two nodes than the concatenation operation.

D. Hyperparameter Investigation (RQ3)

In this section, we will conduct several configuration investigations for our BaGFN model with different hyperparameters. Specifically, we investigate the following crucial factors: 1) different epochs; 2) diverse embedding dimensions; 3) iteration steps of multihop feature fusion; 4) ablation study of attentional components; and 5) broad attentive weights.

1) *Different Epochs*: Epoch is a crucial parameter for model training. Generally, an epoch refers to the process of sending all data into the network to complete a forward calculation and back propagation. During the training, it is not enough to update network weights by a single epoch. Thus, we need multiple epochs to achieve the fitting and convergence of training. In this article, Figs. 5 and 6 show the prediction performance of high-order models under different epochs during model training. From these figures, we can obtain some important results about the influence of different epochs. First, our BaGFN model can obtain the best prediction performance when its epoch values are respectively fixed to 3, 3, and 10 on Avazu, Criteo, and Tobacco datasets. The main reason is that, with the increase in epoch, the network weights of our BaGFN model are updated iteratively during training, and its performance is continuously improved until convergence. However, the larger epoch cannot always get the better performance because of the overfitting problem, and the optimal epoch values for different datasets are also different due to the convergence speed. Second, compared with the other models, our BaGFN model always possesses the highest AUC and lowest Logloss values under different epochs. The main reason is that our BaGFN model has the more excellent ability of modeling high-order feature interactions. The whole training of our BaGFN model is very stable and effective because all AUC values are increasing and all Logloss values are decreasing on three evaluation datasets. Third, the optimal epoch value on the Tobacco dataset is larger than those on the other two datasets. The main reason is that the number of

TABLE III
PERFORMANCES OF DIFFERENT AGGREGATION FUNCTIONS AND EDGE WEIGHTS IN ATTENTIVE GRAPH FUSION MODULE

Model	Avazu		Criteo		Tobacco	
	AUC	Logloss	AUC	Logloss	AUC	Logloss
$model_{org}$	0.7795	0.3798	0.8086	0.4431	0.8795	0.3408
$model_{first}$	0.7799	0.3798	0.8089	0.4428	0.8798	0.3345
$model_{second}$	0.7794	0.3797	0.8085	0.4430	0.8799	0.3344
$model_{concat}$	0.7800	0.3796	0.8090	0.4427	0.8834	0.3334
$model_{ec}$	0.7794	0.3799	0.8090	0.4426	0.8816	0.3341
$model_{final}$	0.7803	0.3794	0.8094	0.4425	0.8870	0.3329

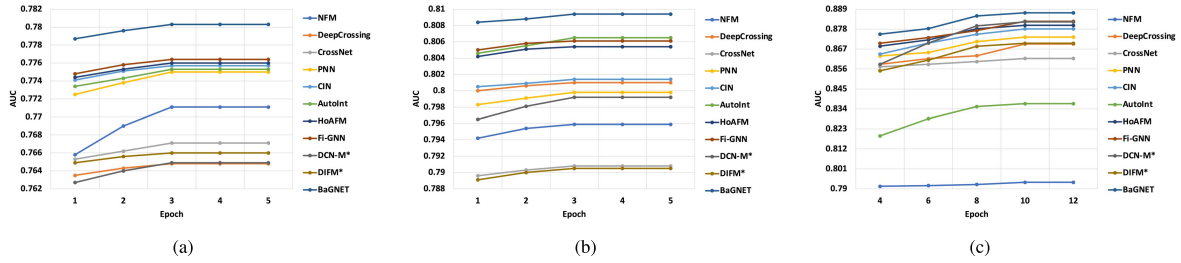


Fig. 5. AUC performance of high-order models for different epochs on (a) Avazu, (b) Criteo, and (c) Tobacco datasets. Note that the baseline models include DeepCrossing [49], CrossNet (Deep&Cross) [2], and PNN [17], and CIN is the core of xDeepFM [10], NFM [8], AutoInt [11], HoAFM [21], Fi-GNN [13], DCN-M* [18], and DIFM* [22]. Compared with the other models, our BaGFN model has the highest AUC values under different epochs. The optimal epochs are respectively fixed to 3, 3, and 10 on three evaluation datasets.

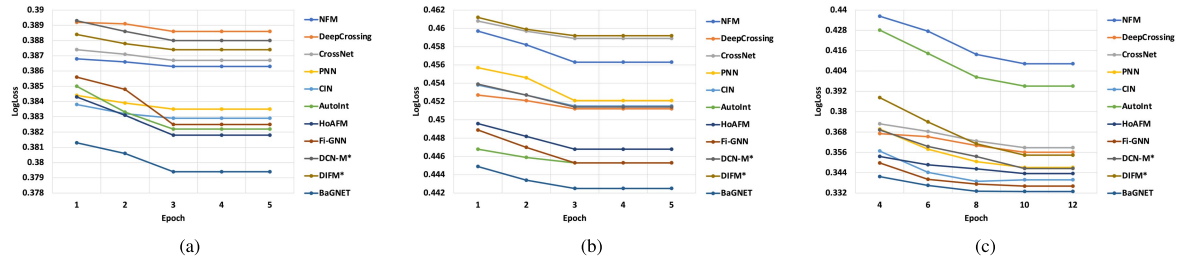


Fig. 6. Logloss performance of high-order models for different epochs on (a) Avazu, (b) Criteo, and (c) Tobacco datasets. Note that the baseline models include DeepCrossing [49], CrossNet (Deep&Cross) [2], and PNN [17], and CIN is the core of xDeepFM [10], NFM [8], AutoInt [11], HoAFM [21], Fi-GNN [13], DCN-M* [18], and DIFM* [22]. Compared with the other models, our BaGFN model has the lowest Logloss values under different epochs. The optimal epochs are respectively fixed to 3, 3, and 10 on three evaluation datasets.

epochs is related to the diversity of the dataset. The Tobacco dataset contains 249 fields, much more than 23 fields on the Avazu dataset and 39 fields on the Criteo dataset. The stronger the diversity is, the larger the epoch value should be. The high-order feature interactions on this dataset are more complex and need more epochs to achieve the convergence of training.

2) *Diverse Embedding Dimensions*: The embedding dimension is of key significance to provide sufficient representation capability. Thus, we explore how the performance of high-order models changes under the different embedding dimensions. As shown in Figs. 7 and 8, since the larger embedding dimensions yield more information, the performance of high-order models can continue to increase when the embedding dimension is below the optimal setting. However, when the embedding dimensions surpass the optimal setting, too much information might cause that the models are overfit, and their prediction performance will decrease. Compared with the other models, our BaGFN model can reach the best prediction performance on three evaluation datasets when all three embedding dimensions are set to 20, which is the optimal setting.

3) *Iteration Steps of Multihop Feature Fusion*: Generally, the iteration steps of multihop feature fusion are constructed to generate a high-order representation for feature interactions. In order to find out the influence of multihop feature fusion, we investigate the performance of our BaGFN model under the different iteration steps. As shown in Fig. 9, the moderate iteration steps can effectively improve the prediction performance on three evaluation datasets. For Avazu and Criteo datasets, our proposed model can reach the best performance when the iteration steps are set to three. For the Tobacco dataset, the optimal iteration step is set to four. Since Avazu and Criteo datasets possess 23 and 39 fields, respectively, three iteration steps are sufficient to generate high-order feature interactions. In contrast, since the Tobacco dataset possesses 249 fields, much more than Avazu and Criteo datasets, more iteration steps are required to adequately interact with other nodes in the graph network.

4) *Ablation Study of Attentional Components*: Although we have yielded powerful empirical results for several attentional components in our BaGFN model, the results have not demonstrated their specific contributions. Therefore, we conduct ablation experiments over our proposed model to better understand

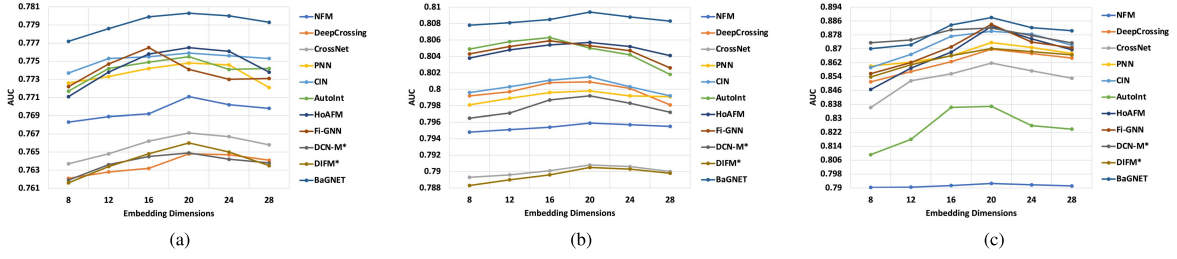


Fig. 7. AUC performance of high-order models for diverse embedding dimensions on (a) Avazu, (b) Criteo, and (c) Tobacco datasets. Note that the baseline models include DeepCrossing [49], CrossNet (Deep&Cross) [2], and PNN [17], and CIN is the core of xDeepFM [10], NFM [8], AutoInt [11], HoAFM [21], Fi-GNN [13], DCN-M* [18], and DIFM* [22]. Compared with the other models, our BaGNET model has the highest AUC values under different embedding dimensions. The optimal embedding dimensions are fixed to 20 on all three evaluation datasets.

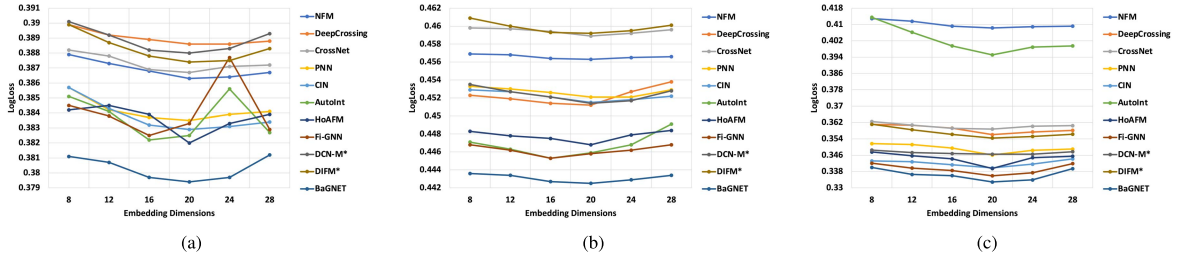


Fig. 8. Logloss performance of high-order models for diverse embedding dimensions on (a) Avazu, (b) Criteo, and (c) Tobacco datasets. Note that the baseline models include DeepCrossing [49], CrossNet (Deep&Cross) [2], and PNN [17], and CIN is the core of xDeepFM [10], NFM [8], AutoInt [11], HoAFM [21], Fi-GNN [13], DCN-M* [18], and DIFM* [22]. Compared with the other models, our BaGNET model has the lowest Logloss values under different embedding dimensions. The optimal embedding dimensions are fixed to 20 on all three evaluation datasets.

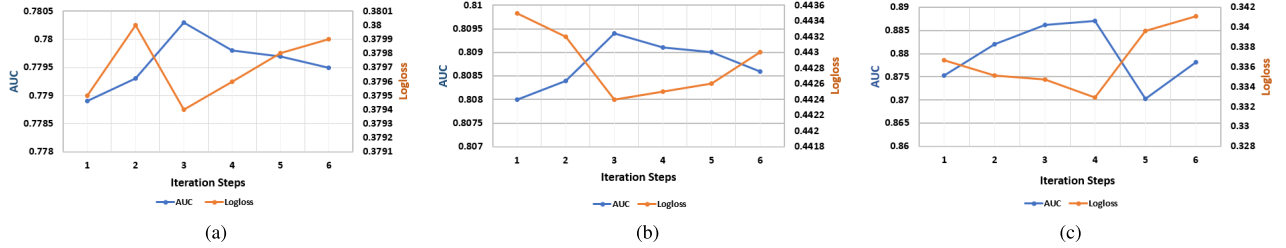


Fig. 9. Performance of our BaGNET model under different iteration steps of multihop feature fusion. For Avazu and Criteo datasets, our proposed model can reach the best prediction performance when the iteration steps are set to three; for the Tobacco dataset, the optimal iteration steps is set to four. (a) Avazu. (b) Criteo. (c) Tobacco.

their relative significance. We define our model with full attentional components as the BASE model and construct two incomplete models by removing the partial attentional components in the following manners.

- 1) *No-BROAD*: Remove the broad attentive cross module from the BASE model.
- 2) *No-SA*: Remove the self-attentive part in the attentive graph fusion module from the BASE model.

We compare the prediction results of the No-BROAD, No-SA, and BASE models on three evaluation datasets, and Table IV illustrates the following observations.

- 1) Compared with the BASE model, the performances of the No-BROAD and No-SA models drop apparently, which demonstrates that two partial attentional components are essential to improve the prediction performance.
- 2) For the No-BROAD and No-SA models, much larger drops appear on the Tobacco dataset compared to the other datasets, which indicates that the attentional components play a greater role on the dataset with more feature fields.

TABLE IV
ABLATION STUDY OF THE PARTIAL ATTENTIONAL COMPONENTS IN OUR BAGFN MODEL

Datasets	Models	AUC	Logloss
Avazu	NO-BROAD	0.7796	0.3799
	NO-SA	0.7801	0.3795
	BASE	0.7803	0.3794
Criteo	NO-BROAD	0.8087	0.4429
	NO-SA	0.8088	0.4427
	BASE	0.8094	0.4425
Tobacco	NO-BROAD	0.8721	0.3367
	NO-SA	0.8715	0.3360
	BASE	0.8870	0.3329

5) *Broad Attentive Weights*: We design the broad attentive cross module for learning the weights of feature nodes, which can reflect the significance of feature fields on the overall prediction result. Fig. 10 shows the heat map of global-level and case-level broad attentive weights on the Avazu dataset, which indicates that the feature fields play different roles in the clicking behaviors. The global indicates a globally

TABLE V
PERFORMANCES OF VARIOUS MODELS AFTER INTEGRATING IMPLICIT FEATURE INTERACTIONS

Model	Avazu		Criteo		Tobacco	
	AUC	Logloss	AUC	Logloss	AUC	Logloss
Wide&Deep (LR) [4]	0.7752	0.3823	0.8028	0.4495	0.8562	0.3655
DeepFM (FM) [9]	0.7755	0.3831	0.8067	0.4448	0.8651	0.3614
Deep&Cross (CrossNet) [2]	0.7735	0.3834	0.8071	0.4449	0.8693	0.3591
xDeepFM (CIN) [10]	0.7771	0.3826	0.8074	0.4443	0.8835	0.3401
BaGFN⁺	0.7809	0.3792	0.8098	0.4421	0.8873	0.3319

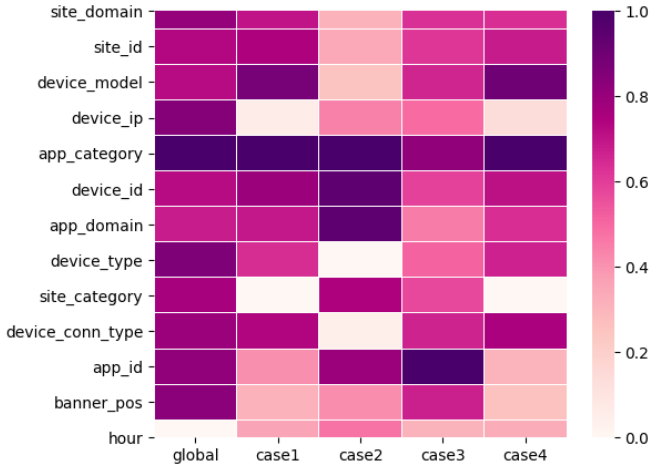


Fig. 10. Heat map of global-level and case-level broad attentive weights on the Avazu dataset, which demonstrates the significance of different feature fields on the final CTR prediction.

averaged one of all samples. Case 1, case 2, case 3, and case 4 are randomly selected from all samples, whose predictive scores are [0.92, 0.15, 0.98, 0.99] and labels are [1, 0, 1, 1], respectively. At the global level, we can find that the feature field *app_category* plays the strongest role in the clicking behavior; at the case level, we observe that, in three of the cases, the feature field *app_category* has the largest influence on predicting the individual clicking behavior. It is reasonable since the Avazu dataset focuses on the mobile scene, where app-related information is a critical factor for CTR prediction.

E. Integrating Implicit Interactions (RQ4)

Feedforward neural networks are able to capture implicit feature interactions and have been extensively integrated into various CTR prediction models [2], [4], [9], [10]. To investigate whether integrating implicit feature interactions further enhances the prediction performance, we combine our BaGFN model with a three-layer feedforward neural network via joint training. Our enhanced model is compared with the following state-of-the-art models.

- 1) *Wide&Deep* [4] combines the outputs of logistic regression and feedforward neural networks.
- 2) *DeepFM* [9] feeds the output of an embedding layer to FM and feedforward neural network, integrating the outputs of the two components to produce the final result.
- 3) *Deep&Cross* [2] is the extension of CrossNet by introducing feedforward neural networks.
- 4) *xDeepFM* [10] is the extension of CIN by integrating feedforward neural networks.

As shown in Table V, based on the prediction performances on three evaluation datasets, we have the following observations.

- 1) All models that integrate implicit interactions clearly boost their prediction ability, which demonstrates the effectiveness of feedforward neural networks.
- 2) Our enhanced model with feedforward neural networks outperforms all other models and achieves the new state-of-the-art performances on three evaluation datasets.

F. Discussion

Generally, the final model prediction accuracy depends on the quality of modeling feature interactions among multi-field sparse data. Our BaGFN model constructs the attentive graph fusion module to aggregate graph node information and strengthen high-order feature representation; furthermore, the broad attentive cross module is designed to learn the significance of different interactions and refine high-order feature interactions at the bitwise level. After a series of experimental comparisons and analyses, our BaGFN model has three main advantages compared with the state-of-the-art methods: 1) the fine-grained cross features are extracted in an explicit manner, which can enhance the model explanations and get rid of negative and uncontrollable feature interactions derived from a rather implicit manner; 2) the multihop feature fusion with bilinear-cross aggregation is more comprehensive and more flexible for sophisticated high-order interactions; and 3) the attentional components explore the importance of different cross features, which can identify the potential concerns of feature interactions more effectively. On the other hand, since the way of feature interaction is more complex, our BaGFN model also has some limitations: 1) more model parameters need to be trained, which consumes more computing resources; 2) for different datasets, manual hyperparameter adjustment is time-consuming and laborious; and 3) multihop feature fusion is easy to suffer from overfitting in model training due to inappropriate iteration steps. In brief, we have conducted a series of experiments to answer four key questions (RQ1–RQ4), and the final responses to these questions have also demonstrated the advantages and disadvantages of our BaGFN model.

V. CONCLUSION

In this article, we establish a novel network named BaGFN as an abbreviation for broad attentive graph fusion network, which aims to integrate graph structure and attention mechanism to generate powerful feature representation and model sophisticated high-order feature interactions among multifield sparse data. Besides the general embedding layer and prediction layer, the broad attentive graph fusion layer is the core of

our BaGFN model. On the one hand, we design a novel attentive graph fusion module in this layer to aggregate graph node information and generate high-order feature representation. On the other hand, we construct a new broad attentive cross module in this layer to highlight the different importance of cross features and refine high-order feature interactions at the bitwise level. Finally, a variety of experimental results on three evaluation datasets demonstrate the excellent performance of our proposed model.

In the future, there are three main directions for better modeling feature interactions. First, our GNN currently focuses on homogeneous graphs with a single node and edge type, i.e., multifield data is defined as a specific type, which usually only requires aggregating a single type of neighbors to update node representation. Next, we will consider heterogeneous graphs with multiple nodes and edge types for more elaborate modeling of feature interactions. Second, temporal prediction analysis inevitably involves features that are dynamically changing over time. However, the current GNN is a static graph mode and lacks full consideration for how efficiently to handle dynamic feature fields. Therefore, we will introduce dynamic graphs for real-time feature change to immediately update node representation of time evolution patterns. Finally, with the success of the Transformer model in machine learning applications, we will also try to integrate the broad attention module in our BaGFN model into the transformer model, such as [51], so that the feature interactions can be captured more efficiently. Besides, the vector of locally aggregated descriptors (VLADs) has been a new trend, and inspired by [52], we will also attempt to further optimize VLAD to achieve better information aggregation for feature interactions.

REFERENCES

- [1] Q. Tan, N. Liu, X. Zhao, H. Yang, J. Zhou, and X. Hu, "Learning to hash with graph neural networks for recommender systems," in *Proc. Int. World Wide Web Conf.*, Apr. 2020, pp. 1988–1998.
- [2] R. Wang, B. Fu, G. Fu, and M. Wang, "Deep & cross network for ad click predictions," in *Proc. ACM SIGKDD Conf. Knowl. Discovery Data Mining*, 2017, pp. 1–7.
- [3] S. C. Geyik, S. Ambler, and K. Kenthapadi, "Fairness-aware ranking in search & recommendation systems with application to linkedin talent search," in *Proc. ACM SIGKDD Conf. Knowl. Discovery Data Mining*, 2019, pp. 2221–2231.
- [4] H.-T. Cheng *et al.*, "Wide & deep learning for recommender systems," in *Proc. Workshop Deep Learn. Rec. Syst.*, 2016, pp. 7–10.
- [5] X. Wang, X. He, F. Feng, L. Nie, and T.-S. Chua, "TEM: Tree-enhanced embedding model for explainable recommendation," in *Proc. World Wide Web Conf.*, 2018, pp. 1543–1552.
- [6] S. Rendle, "Factorization machines," in *Proc. IEEE Int. Conf. Data Mining*, Dec. 2010, pp. 995–1000.
- [7] Y. Juan, Y. Zhuang, W.-S. Chin, and C.-J. Lin, "Field-aware factorization machines for CTR prediction," in *Proc. 10th ACM Conf. Rec. Syst.*, Sep. 2016, pp. 43–50.
- [8] X. He and T.-S. Chua, "Neural factorization machines for sparse predictive analytics," in *Proc. 40th Int. ACM SIGIR Conf. Res. Develop. Inf. Retr.*, Aug. 2017, pp. 355–364.
- [9] H. Guo, R. Tang, Y. Ye, Z. Li, and X. He, "DeepFM: A factorization-machine based neural network for CTR prediction," in *Proc. 26th Int. Joint Conf. Artif. Intell.*, Aug. 2017, pp. 1725–1731.
- [10] J. Lian, X. Zhou, F. Zhang, Z. Chen, X. Xie, and G. Sun, "xDeepFM: Combining explicit and implicit feature interactions for recommender systems," in *Proc. ACM SIGKDD Conf. Knowl. Discovery Data Mining*, 2018, pp. 1754–1763.
- [11] W. Song *et al.*, "AutoInt: Automatic feature interaction learning via self-attentive neural networks," in *Proc. 28th ACM Int. Conf. Inf. Knowl. Manage.*, Nov. 2019, pp. 1161–1170.
- [12] T. Huang, Z. Zhang, and J. Zhang, "FiBiNET: Combining feature importance and bilinear feature interaction for click-through rate prediction," in *Proc. ACM Conf. Rec. Syst.*, 2019, pp. 169–177.
- [13] Z. Li, Z. Cui, S. Wu, X. Zhang, and L. Wang, "Fi-GNN: Modeling feature interactions via graph neural networks for CTR prediction," in *Proc. ACM Int. Conf. Inf. Knowl. Manage.*, 2019, pp. 539–548.
- [14] Y. Li, D. Tarlow, M. Brockschmidt, and R. Zemel, "Gated graph sequence neural networks," in *Proc. Int. Conf. Learn. Represent.*, 2016, pp. 1–20.
- [15] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *Proc. Int. Conf. Learn. Represent.*, 2017, pp. 1–14.
- [16] C. L. P. Chen and Z. L. Liu, "Broad learning system: An effective and efficient incremental learning system without the need for deep architecture," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 29, no. 1, pp. 10–24, Jan. 2018.
- [17] Y. Qu *et al.*, "Product-based neural networks for user response prediction over multi-field categorical data," *ACM Trans. Inf. Syst.*, vol. 37, no. 1, pp. 1–35, Jan. 2019.
- [18] R. Wang *et al.*, "DCN V2: Improved deep & cross network and practical lessons for web-scale learning to rank systems," in *Proc. Int. World Wide Web Conf.*, 2021, pp. 1785–1797.
- [19] W. Cheng, Y. Shen, and L. Huang, "Adaptive factorization network: Learning adaptive-order feature interactions," in *Proc. AAAI Conf. Artif. Intell.*, 2020, vol. 34, no. 4, pp. 3609–3616.
- [20] J. Xiao, H. Ye, X. He, H. Zhang, F. Wu, and T.-S. Chua, "Attentional factorization machines: Learning the weight of feature interactions via attention networks," in *Proc. 26th Int. Joint Conf. Artif. Intell.*, Aug. 2017, pp. 3119–3125.
- [21] Z. Tao, X. Wang, X. He, X. Huang, and T.-S. Chua, "HoAFM: A high-order attentive factorization machine for CTR prediction," *Inf. Process. Manage.*, vol. 57, no. 6, Nov. 2020, Art. no. 102076.
- [22] W. Lu, Y. Yu, Y. Chang, Z. Wang, C. Li, and B. Yuan, "A dual input-aware factorization machine for CTR prediction," in *Proc. 29th Int. Joint Conf. Artif. Intell.*, Jul. 2020, pp. 3139–3145.
- [23] S. Ji, S. Pan, E. Cambria, P. Martinen, and P. S. Yu, "A survey on knowledge graphs: Representation, acquisition, and applications," *IEEE Trans. Neural Netw. Learn. Syst.*, early access, Apr. 26, 2021, doi: 10.1109/TNNLS.2021.3070843.
- [24] B. Xiao, E. R. Hancock, and R. C. Wilson, "Graph characteristics from the heat kernel trace," *Pattern Recognit.*, vol. 42, no. 11, pp. 2589–2606, 2009.
- [25] L. Han, R. C. Wilson, and E. R. Hancock, "Generative graph prototypes from information theory," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 37, no. 10, pp. 2013–2027, Oct. 2015.
- [26] L. Bai, L. Cui, L. Rossi, L. Xu, X. Bai, and E. Hancock, "Local-global nested graph kernels using nested complexity traces," *Pattern Recognit. Lett.*, vol. 134, pp. 87–95, Jun. 2020.
- [27] H.-N. Tran and E. Cambria, "A survey of graph processing on graphics processing units," *J. Supercomput.*, vol. 74, no. 5, pp. 2086–2115, May 2018.
- [28] P. Frasconi, M. Gori, and A. Sperduti, "A general framework for adaptive processing of data structures," *IEEE Trans. Neural Netw.*, vol. 9, no. 5, pp. 768–786, Sep. 1998.
- [29] B. Perozzi, R. Al-Rfou, and S. Skiena, "DeepWalk: Online learning of social representations," in *Proc. ACM SIGKDD Conf. Knowl. Discovery Data Mining*, 2014, pp. 701–710.
- [30] J. Tang, M. Qu, M. Wang, M. Zhang, J. Yan, and Q. Mei, "LINE: Large-scale information network embedding," in *Proc. Int. World Wide Web Conf.*, 2015, pp. 1067–1077.
- [31] S. Cavallari, E. Cambria, H. Cai, K. C.-C. Chang, and V. W. Zheng, "Embedding both finite and infinite communities on graphs [application notes]," *IEEE Comput. Intell. Mag.*, vol. 14, no. 3, pp. 39–50, Aug. 2019.
- [32] F. Scarselli, M. Gori, A. C. Tsoi, M. Hagenbuchner, and G. Monfardini, "The graph neural network model," *IEEE Trans. Neural Netw.*, vol. 20, no. 1, pp. 61–80, Jan. 2009.
- [33] W. L. Hamilton, R. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2017, pp. 1025–1035.
- [34] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph attention networks," in *Proc. Int. Conf. Learn. Represent.*, 2018, pp. 1–12.
- [35] F. Feng, X. He, H. Zhang, and T.-S. Chua, "Cross-GCN: Enhancing graph convolutional network with k-order feature interactions," *IEEE Trans. Knowl. Data Eng.*, early access, May 4, 2021, doi: 10.1109/TKDE.2021.3077524.
- [36] L. Shi, Y. Zhang, J. Cheng, and H. Lu, "Skeleton-based action recognition with directed graph neural networks," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2019, pp. 7904–7913.

- [37] L. Wang, Y. Huang, Y. Hou, S. Zhang, and J. Shan, "Graph attention convolution for point cloud semantic segmentation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2019, pp. 10288–10297.
- [38] Z.-M. Chen, X.-S. Wei, P. Wang, and Y. Guo, "Multi-label image recognition with graph convolutional networks," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2019, pp. 5172–5181.
- [39] D. Teney, L. Liu, and A. Van Den Hengel, "Graph-structured representations for visual question answering," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jul. 2017, pp. 3233–3241.
- [40] Y.-H. Pao and Y. Takefuji, "Functional-link net computing: Theory, system architecture, and functionalities," *Computer*, vol. 25, no. 5, pp. 76–79, May 1992.
- [41] C. L. P. Chen and J. Z. Wan, "A rapid learning and dynamic stepwise updating algorithm for flat neural networks and the application to time-series prediction," *IEEE Trans. Syst., Man, Cybern. B, Cybern.*, vol. 29, no. 1, pp. 62–72, Feb. 1999.
- [42] C. L. P. Chen, Z. Liu, and S. Feng, "Universal approximation capability of broad learning system and its structural variations," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 30, no. 4, pp. 1191–1204, Apr. 2019.
- [43] X. Wang, X. He, Y. Cao, M. Liu, and T.-S. Chua, "KGAT: Knowledge graph attention network for recommendation," in *Proc. ACM SIGKDD Conf. Knowl. Discovery Data Mining*, 2019, pp. 950–958.
- [44] X. Yang, L. Prasad, and L. J. Latecki, "Affinity learning with diffusion on tensor product graph," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 35, no. 1, pp. 28–38, Jan. 2013.
- [45] Z. C. Lipton, J. Berkowitz, and C. Elkan, "A critical review of recurrent neural networks for sequence learning," *Comput. Res. Repository*, vol. abs/1506.00019, pp. 1–38, Jun. 2015.
- [46] K. Greff, R. K. Srivastava, J. Koutnik, B. R. Steunebrink, and J. Schmidhuber, "LSTM: A search space odyssey," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 28, no. 10, pp. 2222–2232, Oct. 2017.
- [47] I. Chaturvedi, Y.-S. Ong, I. W. Tsang, R. E. Welsch, and E. Cambria, "Learning word dependencies in text by means of a deep recurrent belief network," *Knowl.-Based Syst.*, vol. 108, pp. 144–154, Sep. 2016.
- [48] H. B. McMahan *et al.*, "Ad click prediction: A view from the trenches," in *Proc. ACM SIGKDD Conf. Knowl. Discovery Data Mining*, 2013, pp. 1222–1230.
- [49] Y. Shan, T. R. Hoens, J. Jiao, H. Wang, D. Yu, and J. Mao, "Deep crossing: Web-scale modeling without manually crafted combinatorial features," in *Proc. ACM SIGKDD Conf. Knowl. Discovery Data Mining*, 2016, pp. 255–262.
- [50] M. Blondel, A. Fujino, N. Ueda, and M. Ishihata, "Higher-order factorization machines," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2016, pp. 3359–3367.
- [51] J. Yu, J. Li, Z. Yu, and Q. Huang, "Multimodal transformer with multi-view visual representation for image captioning," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 30, no. 12, pp. 4467–4480, Dec. 2020.
- [52] J. Yu, C. Zhu, J. Zhang, Q. Huang, and D. Tao, "Spatial pyramid-enhanced NetVLAD with weighted triplet loss for place recognition," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 31, no. 2, pp. 661–674, Feb. 2020.



Zhifeng Xie received the Ph.D. degree in computer application technology from Shanghai Jiao Tong University, Shanghai, China, in 2013.

He was a Research Assistant with the City University of Hong Kong, Hong Kong. He is currently an Associate Professor with the Department of Film and Television Engineering, Shanghai University, Shanghai. His current research interests include image/video processing, broad learning systems, and computer vision.



Wenling Zhang received the B.Eng. degree in computer science and technology from Nantong University, Nantong, China, in 2018. He is currently pursuing the M.Eng. degree with the Department of Film and Television Engineering, Shanghai University, Shanghai, China.

His current research interests include recommendation systems, broad learning systems, and machine learning.



Bin Sheng (Member, IEEE) the Ph.D. degree in computer science and engineering from The Chinese University of Hong Kong, Hong Kong, in 2011.

He is currently a Full Professor with Shanghai Jiao Tong University, Shanghai, China. His current research interests include virtual reality and computer graphics.

Dr. Sheng is an Associate Editor of the IEEE TRANSACTIONS ON CIRCUITS AND SYSTEMS FOR VIDEO TECHNOLOGY.



Ping Li (Member, IEEE) received the Ph.D. degree in computer science and engineering from The Chinese University of Hong Kong, Hong Kong, in 2013.

He is currently a Research Assistant Professor with The Hong Kong Polytechnic University, Hong Kong. He has one image/video processing national invention patent and has excellent research project reported worldwide by *ACM TechNews*. His current research interests include image/video stylization, artistic rendering and synthesis, and creative media.



C. L. Philip Chen (Fellow, IEEE) received the Ph.D. degree in electrical engineering from Purdue University, West Lafayette, IN, USA, in 1988.

He is currently a Chair Professor and the Dean of the School of Computer Science and Engineering, South China University of Technology, Guangzhou, China. Being a Program Evaluator of the Accreditation Board of Engineering and Technology Education (ABET), Baltimore, MD, USA, for computer engineering, electrical engineering, and software engineering programs, he successfully

architects the University of Macau's Engineering and Computer Science Programs receiving accreditations from Washington/Seoul Accord through Hong Kong Institute of Engineers (HKIE), Hong Kong, of which is considered as his utmost contribution in engineering/computer science education for Macau as the former Dean of the Faculty of Science and Technology. His current research interests include systems, cybernetics, and computational intelligence.

Dr. Chen is a fellow of the American Association for the Advancement of Science (AAAS), the International Association for Pattern Recognition (IAPR), Chinese Association of Automation (CAA), and HKIE. He is a member of Academia Europaea (AE), European Academy of Sciences and Arts (EASA), and International Academy of Systems and Cybernetics Science (IASCYS). He received the IEEE Norbert Wiener Award in 2018 for his contribution to systems and cybernetics, and machine learning. He is also a highly cited researcher by Clarivate Analytics in 2018 and 2019. He was a recipient of the 2016 Outstanding Electrical and Computer Engineers Award from his alma mater, Purdue University, in 1988, after he graduated from the University of Michigan at Ann Arbor, Ann Arbor, MI, USA, in 1985. He was the IEEE Systems, Man, and Cybernetics Society President from 2012 to 2013. He was the Chair of TC 9.1 Economic and Business Systems of International Federation of Automatic Control from 2015 to 2017. He is currently a Vice President of CAA. He was the Editor-in-Chief of the IEEE TRANSACTIONS ON SYSTEMS, MAN, AND CYBERNETICS: SYSTEMS from 2014 to 2019. He is currently the Editor-in-Chief of the IEEE TRANSACTIONS ON CYBERNETICS and an Associate Editor of the IEEE TRANSACTIONS ON FUZZY SYSTEMS.